

SUPERIOR DE TECNOLOGIA EM
**Sistemas para
INTERNET**



Desenvolvimento Web III

Aula 03 –Aplicação com Flask: HTML, CSS, Python

Professora: Mariela Mizota Tamada



**INSTITUTO
FEDERAL**
Rondônia

Campus
Porto Velho Zona Norte

- ✓ Na aula anterior criamos um código em Python com rotas
- ✓ Para cada rota definimos uma função def () que será executada ao chamar a rota correspondente.
- ✓ Também vimos o significado do __name__. E para que serve

 aula03_rotas.porta.def.py >  rota2

```
13  @app_Mariela.route('/rota2')
14  def rota2():
15      resposta = "<H3> Essa é outra página da rota 2 <H3>"
16      return resposta
17
18  def saudacoes (nome):
19      return f'Olá, {nome}'
20
21  if __name__ == "__main__" :
22      app_Mariela.run(port = 8000)  #executa aplicação
23
```

ANP

<https://www.youtube.com/watch?v=mafAKQhjTQY>

[Flask] Como integrar HTML no código Flask e trabalhar com templates

- Aula #03

<https://www.youtube.com/watch?v=eVhnZO iqF0>

[Flask] Como enviar valores de variáveis Python para o HTML pelo método render_template]

- Aula #04

- Personalizar com seus dados
- No dicionário colocar outros campos: nome, disciplina, ano
- Pesquisar o significado de renderização? Que significa no contexto de desenvolvimento web

Nos Frameworks tem várias regras para manter o projeto organizado.

Criar a pasta 'template' dentro do projeto.
Nessa pasta colocaremos as páginas HTML

No código em Python teremos

Route a 'rota' dentro da URL
função com **def ()** em cada Route

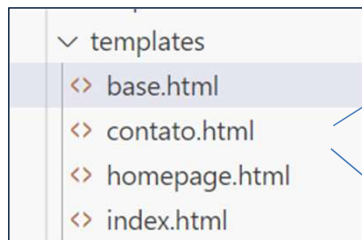
- Colocaremos páginas estáticas.
- Ver como retorna um **render_template** (página.html)
- Que significa *template*?
- Que significa *render* (renderizar)?? Algo estático?
- E para páginas dinâmicas?
- Que significam páginas estáticas e páginas dinâmicas?

Página estática

```
meu_projeto/
├── app.py           # Arquivo principal da aplicação Flask
├── templates/      # Pasta onde ficam os arquivos HTML
│   └── homepage.html # Template HTML que será renderizado
```

- No Flask, os *templates* HTML geralmente ficam em uma pasta chamada “templates”, localizada no mesmo diretório do arquivo principal da aplicação.
- O Flask procura automaticamente na pasta *templates* quando você usa `render_template`, por isso basta colocar os arquivos HTML nessa pasta.

Exercício #1



contato.html

```

1  <!DOCTYPE html>
2  <html lang="pt">
3    <head>
4      <meta charset="utf-8">
5      <title>Contato</title>
6    </head>
7    <body>
8      <h1>Meus dados</h1>
9      <p>e-mail : souts@ifro.edu.br</p>
10     <p>telefone: (69) 3225.4837</p>
11     <p>pix      : souts@ifro.edu.br</p>
12   </body>
13 </html>

```

São dados
estáticos

homepage.html

```

1  <!DOCTYPE html>
2  <html lang="pt">
3    <head>
4      <meta charset="utf-8">
5      <title>Homepage ;) </title>
6    </head>
7    <body>
8      <h2>Seja bem-vindo(a)</h2>
9
10   </body>
11 </html>

```

Exercício #1

LEIA os comentários nos códigos fontes e scripts.

São importantes para compreensão.

Coloque novos comentários para você mesmo lembrar.

Não é só digitar o código. É **compreender** para que você possa **DESENVOLVER** outro diferente.

Observar o uso da
função
render_template ()
que precisa importar e
como é usado no
código

```
5  #nova biblioteca importada chamada render_template
6  from flask import Flask, render_template
7
8  app_Mariela = Flask(__name__ , template_folder='templates')
9  #cria o objeto da aplicação
10
11 @app_Mariela.route("/") #rota para solicitação web
12 def homepage():          #função
13     return render_template ("homepage.html")
14
15 @app_Mariela.route("/contato")
16 def contato():
17     return render_template("contato.html")
18
```

Observar que as
páginas
homepage.html e
contato.html são
páginas estáticas
(slide anterior)

Página dinâmica

A diferença principal entre páginas estáticas e páginas dinâmicas está em como o conteúdo da página é gerado e atualizado.

1. Páginas HTML Estáticas:

- O conteúdo dessas páginas é fixo e pré-definido no arquivo HTML.
- Exibem a mesma informação toda vez que são carregadas, independente de quem acessa ou das interações feitas.
- Geralmente, são arquivos .html prontos, servidos ao cliente sem mudanças no conteúdo.

Exemplos: páginas de políticas de privacidade, arquivos de currículo estáticos.

2. Páginas HTML Dinâmicas:

- O conteúdo é gerado ou modificado com base em variáveis ou interações. Isso pode ocorrer no lado do servidor (como com Flask, Django) ou no lado do cliente (com JavaScript).
- No contexto do Flask: Quando você passa argumentos pelo `render_template`, a página é considerada dinâmica.
- O Flask substitui variáveis e blocos de código condicional, gerando o HTML final que é exibido ao usuário com o conteúdo atualizado.
- Quando você passa dados pela URL, como parâmetros de consulta (e.g., `/?nome=Joao`), e os usa para influenciar a exibição da página, também gera uma página dinâmica, pois o conteúdo muda de acordo com a entrada do usuário.

Exercício #2

```
6  from flask import Flask, render_template
7
8  app_Mariela = Flask(__name__ , template_folder='templates') #cria o objeto da aplicação
9
10 @app_Mariela.route("/index")
11 def indice():
12     return render_template ("index.html")
13
14 @app_Mariela.route("/") #rota para solicitação web
15 def homepage():         #função
16     return render_template ("homepage.html")
17
18 @app_Mariela.route("/contato")
19 def contato():
20     return render_template("contato.html")
21
22 @app_Mariela.route("/usuario")
23 def dados_usuario():
24     nome_usuario="Mariela"
25     dados_usu = {"profissao": "Professora EBTT", "disciplina":"Desenvolvimento Web III"}
26     return render_template("usuario.html", nome = nome_usuario, dados = dados_usu)
27     #parâmetro recebe argumento
28     #colocar o site no ar
29 if __name__ == "__main__":
30     app_Mariela.run(port = 8000)
```

Dicionário para passar
valores ao html

Exercício #2

LEIA os comentários nos códigos fontes e scripts. São importantes para compreensão.

Coloque novos comentários para você mesmo lembrar.

Não é só digitar o código. É **compreender** para que você possa **DESENVOLVER** outro diferente.

Observar o uso da função **render_template ()** que precisa importar e como é usado no código

Aqui vemos algumas das formas para passar argumentos para a página html.

Usamos variável “nome_usuario” e a estrutura de dados dicionário “dados_usu” para passar valores (argumentos) aos parâmetros da função render_template()

```
6  from flask import Flask, render_template
7
8  app_Mariela = Flask(__name__ , template_folder='templates') #cria o objeto da aplicação
9
10 @app_Mariela.route("/index")
11 def indice():
12     return render_template ("index.html")
13
14 @app_Mariela.route("/") #rota para solicitação web
15 def homepage():         #função
16     return render_template ("homepage.html")
17
18 @app_Mariela.route("/contato")
19 def contato():
20     return render_template("contato.html")
21
22 @app_Mariela.route("/usuario")
23 def dados_usuario():
24     nome_usuario="Mariela"
25     dados_usu = {"profissao": "Professora EBTT", "disciplina":"Desenvolvimento Web III"}
26     return render_template("usuario.html", nome = nome_usuario, dados = dados_usu)
27     #parâmetro recebe argumento
28     #colocar o site no ar
29 if __name__ == "__main__":
30     app_Mariela.run(port = 8000)
```

Exercício #2

usuario.html

```
8      <body>
9          <h2>Olá, {{nome}} </h2>
10         <p>
11             <b>Profissão: </b> {{dados['profissao']}} <br/>
12             <b>Disciplina: </b> {{dados['disciplina']}}
13         </p>
14     </body>
```

Acesso aos valores do
dicionário por meio
das chaves para exibir
na tela html

index.html

```
templates > <> index.html > ...
1      <!DOCTYPE html>
2      <html lang="pt">
3          <head>
4              <meta charset="utf-8">
5              <title>Perfil do Usuário</title>
6          </head>
7          <body>
8              <nav>
9                  <a href="/index"> Página Inicial</a>
10                 <a href="/contato"> Contatos</a>
11                 <a href="/usuario"> Perfil </a>
12             </nav>
13             <h2>Seja bem-vindo(a) {{nome}}</h2>
14         </body>
15     </html>
```

href = referência
(a outras páginas)

templates

- <> base.html
- <> contato.html
- <> homepage_nome.html
- <> homepage.html
- <> index.html
- <> testes.html
- <> usuario.html

Exercício #2

- ✓ templates
 - <> base.html
 - <> contato.html
 - <> homepage_nome.html
 - <> homepage.html
 - <> index.html

```
@app_Mariela.route("/index")
def indice():
    return render_template ("index.html")
```

index.html

```
templates > <> index.html > html > body > nav
1  <!DOCTYPE html>
2  <html lang="pt">
3  <head>
4  <meta charset="utf-8">
5  <title>Perfil do Usuário</title>
6  </head>
7  <body>
8  <nav>
9  <a href="/index"> Página Inicial</a>
10 <a href="/contato"> Contatos</a>
11 <a href="/usuario"> Perfil </a>
12 </nav>
13 <h2>Seja bem-vindo(a) {{nome}}</h2>
14
15 </body>
16 </html>
```

← → ↻ ⓘ 127.0.0.1:8000/index

[Página Inicial](#) [Contatos](#) [Perfil](#)

Seja bem-vindo(a)

Página dinâmica via URL

Exercício #3

```
▼ templates
  <> base.html
  <> contato.html
  <> homepage_nome.html
  <> homepage.html
  <> index.html
```

**Passar dados
dinamicamente
pela URL**

```
#####
```

```
"""
```

Página dinâmica. Precisa de 3 elementos:
o dado dinâmico na URL entre < >,
os parâmetros na função e
a página montada dinamicamente.

```
"""
```

```
@app_Mariela.route('/<id>')
```

```
def saudacao(id):
```

```
    return render_template('homepage_nome.html', nome=id)
```

homepage_nome.html

```
templates > <> homepage_nome.html > ...
```

```
1  <!DOCTYPE html>
2  <html lang="pt">
3      <head>
4          <meta charset="utf-8">
5          <title>Homepage ;) </title>
6      </head>
7      <body>
8          <h3>Olá, {{ nome }}. Seja bem-vindo(a)!</h3>
9
10     </body>
11 </html>
```

Pode ser outro nome
no lugar de <id>

Exercício #3

```
"""
Página dinâmica com argumentos via URL. Precisa de 3 elementos:
o dado dinâmico na URL entre < >,
os parâmetros na função e
a página montada dinamicamente.
"""

@app_Mariela.route('/<id>')
def saudacao(id):
    return render_template('homepage_nome.html', nome=id)

@app_Mariela.route("/usuario/<nome_usuario>;<nome_profissao>;<nome_disciplina>")
#o que está entre < > é o dado dinâmico que vai diferenciar a página para cada usuário.
def usuario (nome_usuario, nome_profissao, nome_disciplina): #passa o valor da variável como argumento na função

    dados_usu = {"profissao": nome_profissao, "disciplina": nome_disciplina}
    #dados_usu é um dicionário com 2 chaves:valor

    return render_template ("usuario.html", nome=nome_usuario, dados = dados_usu)
    #o parâmetro 'nome' recebe como argumento o valor armazenado na variável 'nome_usuario'
    #o parâmetro 'dados' recebe como argumento o dicionário dados_usu
```

Exercício #3

LEIA os comentários nos códigos fontes e scripts. São importantes para compreensão.

Coloque novos comentários para você mesmo lembrar.

Não é só digitar o código. É **compreender** para que você possa **DESENVOLVER** outro diferente.

ANTES
(exercício anterior)

```
@app_Mariela.route("/usuario") ←
def dados_usuario():
    nome_usuario="Mariela"
    dados_usu = {"profissao": "Professora EBTT", "disciplina":"Desenvolvimento Web III"}
    return render_template("usuario.html", nome = nome_usuario, dados = dados_usu)
```

DEPOIS

```
@app_Mariela.route("/usuario/<nome_usuario>;<nome_profissao>;<nome_disciplina>")
#o que está entre < > é o dado dinâmico que vai diferenciar a página para cada usuário.
def usuario (nome_usuario, nome_profissao, nome_disciplina): #passa o valor da variável como argumento

    dados_usu = {"profissao": nome_profissao, "disciplina": nome_disciplina}
    #dados_usu é um dicionário com 2 chaves:valor

    return render_template ("usuario.html", nome=nome_usuario, dados = dados_usu)
    #o parâmetro 'nome' recebe como argumento o valor armazenado na variável 'nome_usuario'
    #o parâmetro 'dados' recebe como argumento o dicionário dados_usu
```

Templates

(modelos)
'herança' de template

```
meu_projeto/
├── app.py           # Arquivo principal da aplicação Flask
├── t_templates/    # Pasta onde ficam os arquivos HTML
│   └── homepage.html # Template HTML que será renderizado
```

o Flask já procura automaticamente na pasta `templates`.

No entanto, se você precisar usar um diretório diferente para templates, pode configurar isso manualmente ao instanciar o Flask:

```
app_Mariela = Flask(__name__, template_folder='t_templates')
# os templates coloca em outra pasta.
# Por padrão, fica na pasta templates e não precisa informar no template_folder,
# mas se quiser armazenar em outra pasta indique nesse parâmetro.
```

ANP

<https://www.youtube.com/watch?v=epTuwBcfjJk>

[Flask] Criando um template base para várias páginas HTML em Flask (herança de templates) - Aula #05

A **herança de templates no Flask** é um recurso poderoso do **Jinja2**, o mecanismo de template usado pelo Flask.

Ele permite que você **defina uma estrutura base para suas páginas HTML** (como cabeçalhos, rodapés e menus) e personalize partes específicas em diferentes páginas usando blocos.

A herança de templates funciona com **{% block %}** criando um template base.

Você cria um arquivo **base.html** com a estrutura geral do site, deixando espaços para personalização com **{% block nome%} {%endblock%}**.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Título Padrão{% endblock %}</title>
</head>
<body>
  <header>
    <h1>Meu Site</h1>
    <nav>
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/about">Sobre</a></li>
      </ul>
    </nav>
  </header>
  <main>
    {% block content %}
    <!-- Conteúdo principal será inserido aqui -->
    {% endblock %}
  </main>
  <footer>
    <p>&copy; 2024 Meu Site</p>
  </footer>
</body>
</html>
```

cria outros templates que estendem o **base.html** e **preenchem** ou **sobrescrevem** os blocos definidos.

base.html

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport", initial-scale=1.0">
  <title>{% block title %} Título Padrão {% endblock %}</title>
</head>
```

sobrescreve

```
<main>
  {% block content %}

  {% endblock %}
</main>
```

home.html

```
{% extends "base.html" %}

{% block title %}Página Inicial{% endblock %}

{% block content %}
  <h2>Bem-vindo ao meu site!</h2>
  <p>Este é o conteúdo da página inicial.</p>
{% endblock %}
```

about.html

```
{% extends "base.html" %}

{% block title %}Sobre Nós{% endblock %}

{% block content %}
  <h2>Sobre Nós</h2>
  <p>Informações sobre a empresa.</p>
{% endblock %}
```

Exercício #4

a diretiva `{% extends 'base.html' %}` e os blocos de conteúdo que devem ser circundados pelas diretivas `{% block conteudo %}` e `{% endblock %}`.

O método `render_template` então é capaz de substituir os trechos únicos a partir da página com conteúdo comum a todas as demais.

Então, dentro do código em Python teremos mais um elemento

- # Route a 'rota' dentro da URL
- # função com `def ()` em cada Route
- # templates

Exercício #4

✓ t_templates
 <> base.html
 <> t_cadastro.html
 <> t_contato.html
 <> t_index.html
 <> t_login_flash_js_cad
 <> t_login_flash_js.htm
 <> t_login_flash.html
 <> t_login.html
 <> t_usuario.html

base.html

Link copiado da
imagem em
ifro.edu.br



```
1  <!DOCTYPE html>
2  <!-- página template(modelo) com layout base das páginas do site -->
3
4  <html lang="pt">
5    <head>
6      <meta charset="utf-8">
7      <title>Perfil do Usuário</title>
8    </head>
9    <body>
10     <!--Informação fixa nas páginas coloca neste template -->
11     
12     <br>
13     <nav>
14       <a href="/index">Página Inicial</a>
15       <a href="/contato">Contatos</a>
16     </nav>
17
18     <!--Informação que muda nas páginas coloca como extensão -->
19     {% block conteudo %}
20
21     {% endblock %}
22
23     <h6>Copyright© 2023</h6> <!--Ctrl+Alt+C-->
24   </body>
25 </html>
```

Adotei o prefixo **t_** para diferenciar do exercício anterior.

```
▼ t_templates
  <> base.html
  <> t_cadastro.html
  <> t_contato.html
  <> t_index.html
  <> t_login_flash_js_cad
  <> t_login_flash_js.htm
  <> t_login_flash.html
  <> t_login.html
  <> t_usuario.html
```

t_contato.html

```
1  {% extends 'base.html' %}
2  <!-- Diretiva do Flask para indicar que esta
3
4  {% block conteudo %}
5      <h1>Meus dados</h1>
6      <p>e-mail : souts@ifro.edu.br</p>
7      <p>telefone: (69) 3225.4837</p>
8      <p>pix      : souts@ifro.edu.br</p>
9
10 {% endblock %}
```

t_index.html

```
1  {% extends 'base.html' %}
2  <!-- Diretiva do Flask para indicar que es
3
4  {% block conteudo %}
5
6  <h2>Seja bem-vindo(a) {{nome}}</h2>
7
8  {% endblock %}
```

t_usuario.html

```
t_templates > <> t_usuario.html > ...
1  {% extends 'base.html' %}
2
3  {% block conteudo %}
4
5  <h2>Olá, {{nome}} </h2>
6  <p>
7      <br>
8      <b>Profissão: </b> {{ dados['profissao'] }} <br/>
9      <b>Disciplina: </b> {{ dados['disciplina'] }}
10 </p>
11
12 {% endblock %}
```

LEIA os comentários nos códigos fontes e scripts. São importantes para compreensão.

Coloque novos comentários para você mesmo lembrar.

Não é só digitar o código. É **compreender** para que você possa **DESENVOLVER** outro diferente.

Existem 3 **routes** (rotas) para executar. O que será executado está dentro da função **def()**.

No exemplo, cada def chama uma página **html**.

```
app_Mariela = Flask(__name__, template_folder='t_templates')
# os templates coloca em outra pasta.
# Por padrão, fica na pasta templates e não precisa informar no template_folder,
# mas se quiser armazenar em outra pasta indique nesse parâmetro.

@app_Mariela.route("/")          #se no navegador digitar / ou /index
@app_Mariela.route("/index")
def indice():
    return render_template("t_index.html") #optei por prefixar com t_ os nomes dos arquivos

@app_Mariela.route("/contato")
def contato():
    return render_template("t_contato.html")

@app_Mariela.route("/usuario", defaults={"nome_usuario":"usuário?","nome_profissao":""})
def usuarios (nome_usuario, nome_profissao):
    dados_usu = {"profissao": nome_profissao, "disciplina":"Desenvolvimento Web III"}
    return render_template("t_usuario.html", nome=nome_usuario, dados = dados_usu)

if __name__ == "__main__":
    app_Mariela.run(port = 8000)
```

Adotei o prefixo t_ para diferenciar do exercício anterior.

Vantagens da Herança de Templates:

- ✓ **Reutilização de código:** Evita duplicação ao manter cabeçalhos, rodapés e menus em um único arquivo.
- ✓ **Organização:** Facilita a manutenção e leitura do código.
- ✓ **Flexibilidade:** Blocos permitem personalização sem alterar a estrutura principal.

Incluindo arquivos estáticos

(imagens e CSS)

Exercício #5 Imagens estáticas e CSS

estilo.css

NEW

```
Projetos > flaskweb > static > CSS > # estilo.css >
1  .imgquadrada{
2      width: 70px;
3      height: 70px;
4  }
5
```

base.html

```
4  <html lang="pt">
5      <head>
6          <meta charset="utf-8">
7          <title>Perfil do Usuário</title>
8          <link rel = "stylesheet" href="{{ url_for('static', filename = 'css/estilo.css')}}"
9      </head>
10     <body>
11         <div>
12             <!--Informação fixa nas páginas coloca neste template -->
13             
15         </div>
16
```

t_usuario.html

```
t_templates > <> t_usuario.html > ...
1  {% extends 'base.html' %}
2
3  {% block conteudo %}
4
5      <h2>Olá, {{nome}} </h2>
6      <p>
7          <!-- 'static' é a pasta do projeto onde copio as imagens -->
8          <img class="imgquadrada" src = "{{ url_for ('static', filename = 'imagens/professora.jpg') }}" >
9
10         <br>
11         <b>Profissão: </b> {{ dados['profissao'] }} <br/>
12         <b>Disciplina: </b> {{ dados['disciplina'] }}
13     </p>
14
15     {% endblock %}
```

<https://www.youtube.com/watch?v=9BBviKJsGb0>

[Flask] Incluindo arquivos estáticos (imagens e css) no seu projeto - Aula #07

<https://www.youtube.com/watch?v=nmofyhEcIPg>

[Flask] Trabalhando com passagem de parâmetros via rotas - Aula #08

<https://www.youtube.com/watch?v=kxLirPym7mU>

[Flask] Passagem de parâmetros via métodos GET - Aula #09

Passagem de argumentos default (padrão) na URL

Ao acessar o menu temos só 2 páginas,
Página Inicial
que correspondem ao `route("/index")`

E **página de contato**
Que corresponde ao `route("/contato")`

Informado no código em Python

```
@app_Mariela.route("/") #se no navegador digitar / ou /index
@app_Mariela.route("/index")
def indice():
    return render_template("t_index.html")

@app_Mariela.route("/contato")
def contato():
    return render_template("t_contato.html")

#rota /usuarios COM passagem de argumentos
@app_Mariela.route("/usuarios/<nome_usuario>;<nome_profissao>")
```

```
<!-- página template(modelo) com layout base das páginas do site -->
4 <html lang="pt">
5   <head>
6     <meta charset="utf-8">
7     <title>Perfil do Usuário</title>
8   </head>
9   <body>
10    <!--Informação fixa nas páginas coloca neste template -->
11    
13    <nav>
14      <a href="/index">Página Inicial</a>
15      <a href="/contato">Contatos</a>
16    </nav>
17
18    <!--Informação que muda nas páginas coloca como extensão -->
19    {% block conteúdo %}
```

base.html

Como acessar ao 3º. route (/usuários) que espera 2 argumentos (valores) para os parâmetros nome e profissão ?

Se não informar esses parâmetros
dá erro !!
Testem e verifiquem!

Exercício #5

Trabalhando com passagem de argumentos via rotas

base.html

Observe que posso definir argumento **default** (padrão), caso o argumento não seja informado na URL.

```
<nav>
  <a href="/index">Página Inicial</a>
  <a href="/contato">Contatos</a>
  <a href="/usuarios">Perfil do Usuário</a>
</nav>
```

← → ↻ ⓘ 127.0.0.1:8080/usuarios



[Página Inicial](#) [Contatos](#) [Perfil do Usuário](#)

#rota /usuarios COM passagem de argumentos

```
@app_Mariela.route("/usuarios/<nome_usuario>;<nome_profissao>")
```

#rota /usuarios SEM passagem de argumentos --> definir valor padrão com defaults

```
@app_Mariela.route("/usuarios", defaults={"nome_usuario":"usuário?", "nome_profissao":""})
```

```
def usuarios (nome_usuario, nome_profissao):
```

```
    dados_usu = {"profissao": nome_profissao, "disciplina":"Desenvolvimento Web III"}
```

```
    return render_template ("t_usuario.html", nome=nome_usuario, dados = dados_usu)
```

Olá, usuário?



Métodos Get e Post

Por padrão, o navegador usa o método **GET** se o atributo *method* não estiver presente.

O formulário está configurado para usar o método GET. Então, se desejar o funcionamento como POST o atributo *method*="**POST**" deve ser especificado no elemento **<form>**.

```
<form action="/autenticar" method="POST" >
  <!-- ao clicar no botão OK, no submit vai para a rota /autenticar -->
  <!-- Quando trabalha com métodos GET (padrão) ou POST (declara method) -->
  <br/><br/>

  Usuário: <input type = "text" name="nome_usuario"><br/><br/>
  Senha  : <input type = "text" name="senha"><br/><br/>
  <button type = "submit">OK</button>
</form>
```

```
@app_Mariela.route("/login")
def login():
    return render_template("t_login.html")
```

NEW

t_login.html

Por padrão é método **GET**. Por isso é opcional colocar **<form action=/autenticar method="GET" >**

Projetos > flaskweb > templates > <> t_login.html > ...

```
1 {% extends 'base.html' %}
2
3 {% block conteudo %}
4     <form action="/autenticar">
5         <!-- ao clicar no botão OK, no submit vai para a rota /autenticar -->
6         <br/><br/>
7         <!--Quando trabalha com métodos GET (padrão) ou POST definimos
8             os nomes e tipo de dado de cada campo que trabalha no formulário-->
9         Usuário: <input type = "text" name="nome_usuario"><br/><br/>
10        Senha  : <input type = "text" name="senha"><br/><br/>
11        <button type = "submit">OK</button>
12    </form>
13
14 {% endblock %}
```

← → ↻ ⓘ 127.0.0.1:8080/autenticar?nome_usuario=&senha=

Not Found

The requested URL was not found on the server.
again.

Exercício #6

Passagem de parâmetros via métodos **GET**



```
from flask import request  
#adicionar para trabalhar com os métodos GET e POST
```

➔ #para poder recuperar os argumentos passados nos parâmetros na URL precisa importar o pacote
#from flask import request
também precisa colocar que essa página aceita requisições de tipo GET
@meu_site.route("/autenticar", methods=['GET', 'POST'])
def autenticar():
 usuario = request.args.get('nome_usuario')
 senha = request.args.get('senha')
 return f"usuario: {usuario} e senha: {senha}"

127.0.0.1:8080/login

INSTITUTO FEDERAL Rondônia

[Página Inicial](#) [Contatos](#) [Perfil do Usuário](#)

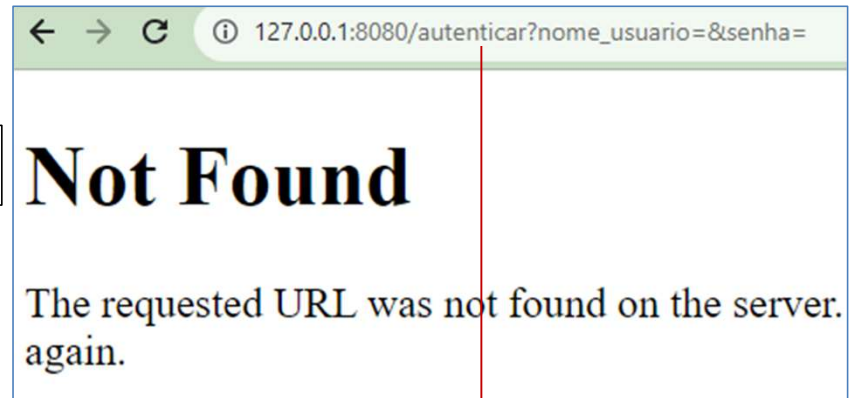
Usuário:

Senha :

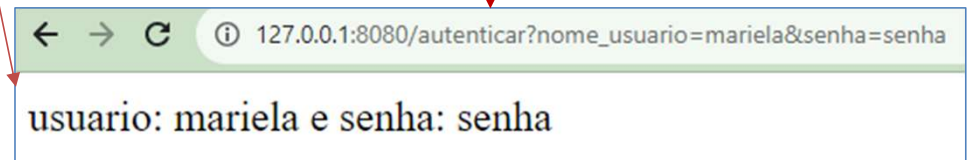
OK

Copyright© 2023

ANTES



DEPOIS



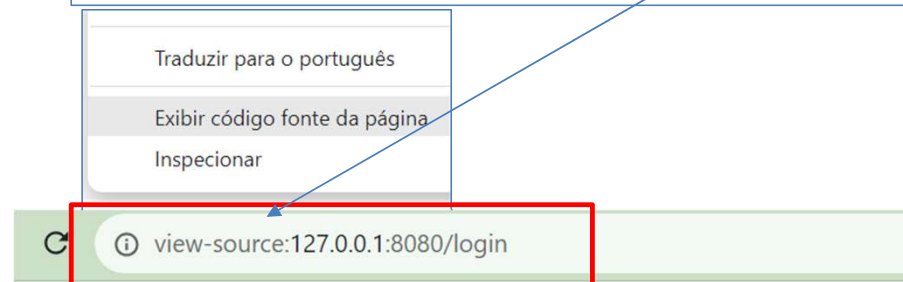
Exercício #7 Passagem de parâmetros via método **POST**

t_login.html

É necessário colocar o **method="POST"**

```
rojetos > flaskweb > templates > <> t_login.html > form
1  {% extends 'base.html' %}
2
3  {% block conteudo %}
4      <form action="/autenticar" method="POST">
5          <!-- ao clicar no botão OK, no submit vai para a rota /
6          <br/><br/>
7          <!--Quando trabalha com métodos GET (padrão) ou POST def
8          | os nomes e tipo de dado de cada campo que trabalha r
9          Usuário: <input type = "text" name="nome_usuario"><br/><
10         Senha  : <input type = "text" name="senha"><br/><br/>
11         <button type = "submit">OK</button>
12     </form>
13
14 {% endblock %}
```

Na página html clique direito e selecione “Exibir Código”, vai ver que no **view-source** mostra o método



</div>

<nav>

Página Inicial

Contatos

Perfil do Usuário

</nav>

<!--Informação que muda nas páginas coloca co

<form action="/autenticar" method="POST">

<!-- ao clicar no botão OK, no submit va

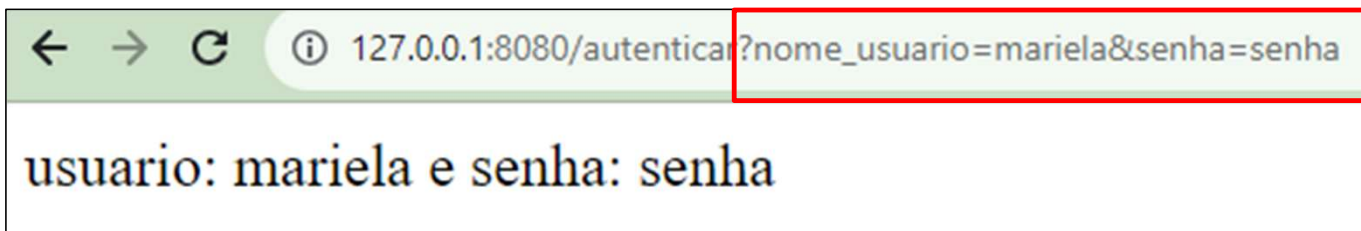
<!--Quando trabalha com métodos GET (padr



Diferença entre os métodos GET e POST

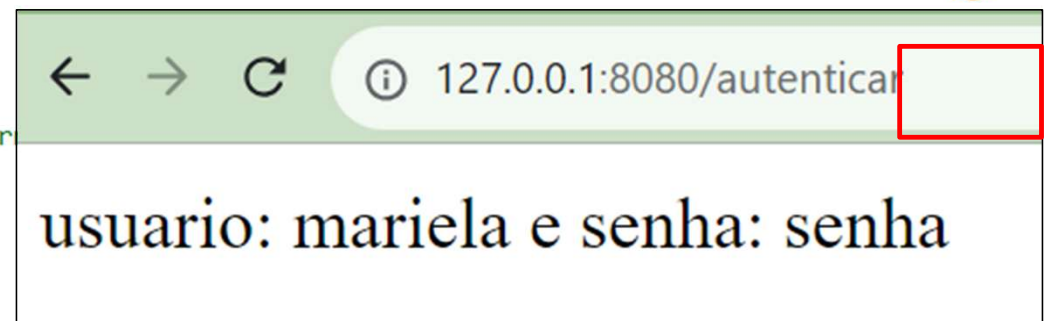
```
@app_Mariela.route("/autenticar", methods=['GET', 'POST'])
def autenticar():
    #método GET - recebe pela URL ==> (args)
    usuario = request.args.get('nome_usuario')
    senha = request.args.get('senha')
    return f"usuario: {usuario} e senha: {senha}"
```

Método GET (args)

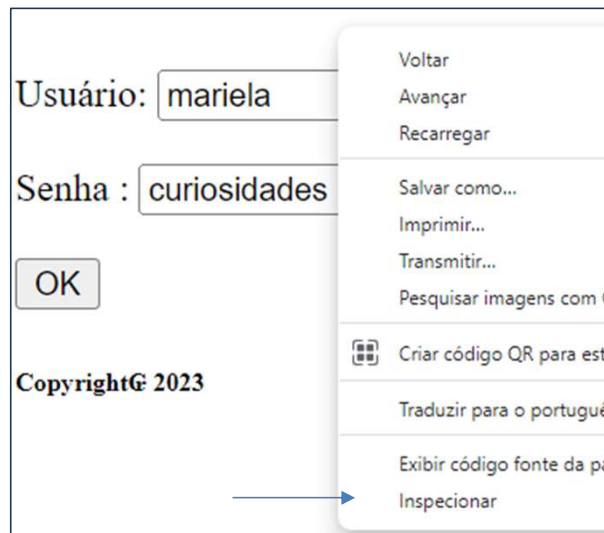


Método POST (form)

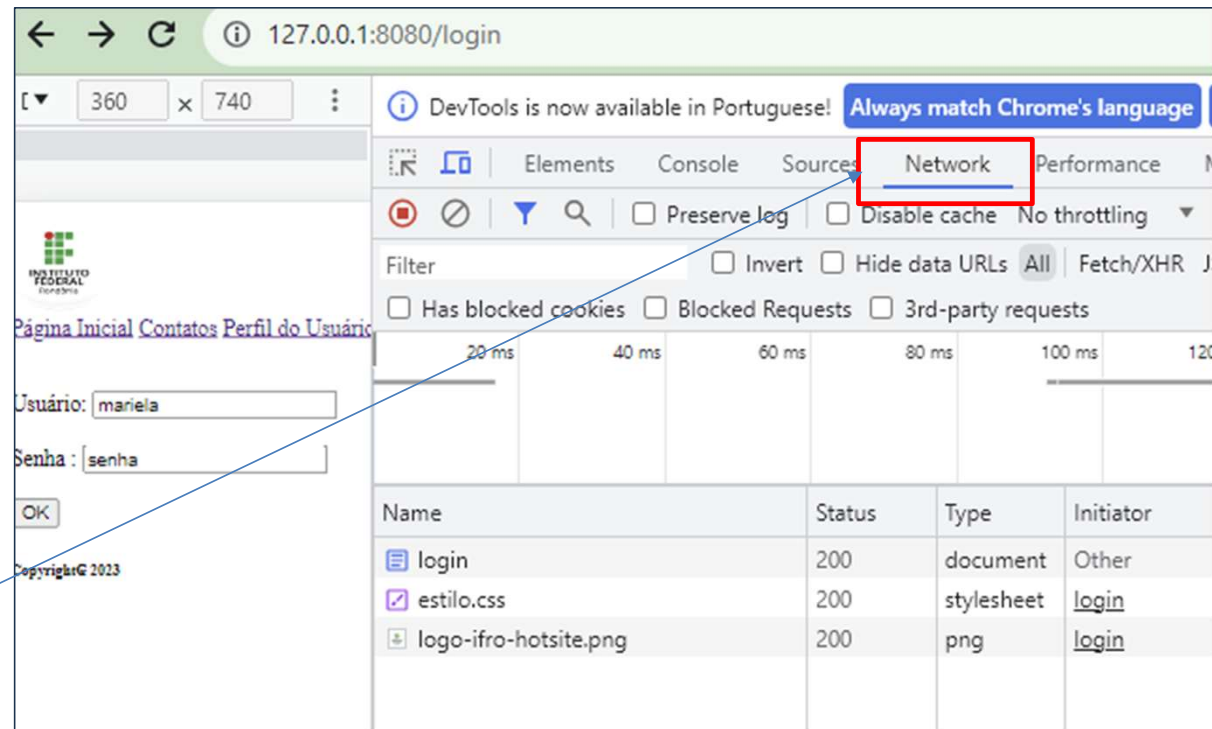
```
.....
@app_Mariela.route("/autenticar", methods=['GET', 'POST'])
def autenticar():
    #método POST - pega nos fields (campos) do formulário => for
    usuario = request.form.get('nome_usuario')
    senha = request.form.get('senha')
    return f"usuario: {usuario} e senha: {senha}"
```



Para ver no navegador como funciona o POST



Clique direito →
“Inspecionar”



Para ver no navegador como funciona o POST

Para visualizar os dados enviados em um formulário HTML ao usar o método POST, você deve seguir estes passos:

1. Abrir a Ferramenta de Desenvolvimento no navegador. Clique com o botão direito na página e escolha **"Inspecionar"**
2. Ir para a Aba Network: Clique na aba **"Network"** na parte superior da ferramenta.
3. Enviar o Formulário: Preencha os campos do formulário e clique no botão de envio para fazer a requisição. Certifique-se de que o método POST está configurado corretamente.
4. Selecionar a Requisição: Na aba "Network", você verá **uma lista de requisições** feitas pela página. Encontre a requisição que aponta para o *endpoint* configurado, por exemplo, /autenticar.
5. Visualizar os Dados Enviados: Clique na requisição para abri-la. Vá até a aba **"Payload"** (localizada ao lado de "Headers"). Na seção **Form Data**, você verá as variáveis e valores enviados no formulário.

The screenshot shows the Chrome DevTools Network tab. The address bar at the top displays the URL `127.0.0.1:8080/autenticar`, which is highlighted with a red box. Below the address bar, the 'Network' tab is selected, also highlighted with a red box. The 'Filter' section shows 'All' selected. The 'Has blocked cookies' checkbox is checked. The 'Blocked Requests' and '3rd-party requests' checkboxes are unchecked. The 'Timeline' view shows a request bar for 'autenticar' at approximately 20 ms. The 'Name' column in the list of requests shows 'autenticar', which is highlighted with a red box. The 'Form Data' tab is selected, showing the following data:

| Form Data | view source | view URL-encoded |
|-----------------------|-------------|------------------|
| nome_usuario: mariela | | |
| senha: curiosidades | | |

Two blue arrows point from the text 'usuario: mariela e senha: curiosidades' to the 'nome_usuario: mariela' and 'senha: curiosidades' entries in the 'Form Data' tab.

Nome dos parâmetros e seus
argumentos (valores)

Não visualizou? Verifique:

Se a lista de requisições não aparece na aba Network, pode ser por algum dos seguintes motivos.

1. Certifique-se de que está na aba certa: Ao clicar em Network, a área deve começar a registrar todas as requisições feitas pela página.
2. Habilite o Monitoramento de Requisições: Às vezes, o monitoramento está pausado. Certifique-se de que o ícone de "**gravação**" no canto superior esquerdo da aba Network (um círculo vermelho) está ativo. Se não estiver, clique nele para ativar.
3. Recarregue a Página: Depois de ativar o monitoramento, recarregue a página (F5) para capturar todas as requisições feitas ao enviar o formulário.
4. Filtre por Tipo de Requisição: Certifique-se de que o filtro correto está ativo. Por exemplo: **Selecione "All"** (todas as requisições). Ou clique em "XHR", que geralmente mostra as requisições POST ou GET.
5. Envie o Formulário Novamente: Após garantir que a aba Network está ativa, preencha o formulário e clique no botão de envio. A requisição deve aparecer na lista.

Resumo – Diferença entre GET e POST



A principal diferença entre GET e POST é que:

- ✓ o GET é usado **para solicitar** dados do servidor e os **parâmetros são visíveis na URL**
- ✓ o POST é usado **para enviar** dados ao servidor **no corpo da solicitação** (não visíveis na URL) e geralmente é usado para realizar ações que alteram o estado do servidor.

A escolha entre GET e POST depende do tipo de operação que você deseja realizar em seu aplicativo web.

API?

Relação com Rotas (*routes*)

"Toda API é uma rota, mas nem toda rota é uma API."

Toda rota é um *endpoint* acessível via URL.

Elas podem retornar **qualquer tipo de conteúdo**: HTML, JSON, XML, texto, etc.

```
from flask import Flask

app = Flask(__name__)

# Esta é uma rota que retorna HTML (uma página web)
@app.route('/sobre')
def sobre():
    return '<h1>Página Sobre</h1>'

# Esta é uma rota que retorna dados (uma API)
@app.route('/api/usuarios')
def obter_usuarios():
    return {'usuarios': ['Ana', 'João', 'Maria']}
```

O que é uma API (Application Programming Interface)?

Uma API é um conjunto de regras e protocolos que permite que diferentes softwares se comuniquem.

No contexto web (API RESTful), são rotas projetadas especificamente para **trocar dados estruturados** (geralmente JSON) entre uma aplicação (ex: um front-end React) e um servidor, em vez de retornar HTML para um navegador.

Características de uma Rota que é uma **API**:

- Retorna Dados Estruturados: JSON é o formato mais comum.
- Operações CRUD: São mapeadas para os verbos HTTP (GET, POST, PUT, DELETE).
- GET /api/produtos -> Obtém a lista de produtos.
- POST /api/produtos -> Cria um novo produto.
- *Stateless*: Cada requisição deve conter toda a informação necessária para ser processada.
- Consumida por Código: É feita para ser lida por outras aplicações ou serviços, não diretamente por um usuário final em um navegador.

Conclusão

“Toda API é uma rota”: Para que uma API seja acessível via web, ela deve ser exposta através de uma URL, ou seja, de uma rota. GET /api/usuarios é uma rota que é também um *endpoint* de API.

“Mas nem toda rota é uma API”: Uma rota como /sobre que retorna uma página HTML completa para ser visualizada por um humano não é considerada uma API. Seu propósito é diferente.

O termo "rota" é o conceito mais amplo (qualquer URL do seu app).

O termo "API" é um subtipo específico de rota, especializado em programação e troca de dados entre máquinas.

Essa distinção é crucial para arquitetar aplicações modernas, onde o back-end (Flask) often atua como uma API RESTful para um front-end (React, Vue.js) separado.

Flash
(mensagens enviadas
pelo back)

```
from flask import Flask, render_template
from flask import request    #para trabalhar com os métodos GET e POST
from flask import flash      #para msgs popup
from flask import redirect    #para redirecionar páginas
```

```
app_Mariela = Flask(__name__, template_folder='t_templates')
# no caso de usar flash pede a configuração de uma chave secreta
app_Mariela.config['SECRET_KEY'] = "palavra-secreta-IFRO"
```

Exercício #8

main.py

```
@app_Mariela.route("/autenticar", methods=['GET', 'POST'])
def autenticar():
    #método POST - pega nos fields (campos) do formulário
    usuario = request.form.get('nome_usuario')
    senha = request.form.get('senha')
    if usuario == "admin" and senha == "ifro":
        return f"usuario: {usuario} e senha: {senha}"
    else:
        #para não dar msg. na outra página, vamos manter na própria página com flash
        #adicionar import flash
        flash("Dados inválidos!")
        flash("Login ou senha inválidos!")
        return redirect ('/login') #adicionar import redirect
```

LEIA !

redireciona para a rota /login
Ou seja vai voltar a chamar a rota e ficar na tela login.htm
que já estava.

Exercício #8

main.py

```
@app_Mariela.route("/autenticar", methods=['GET', 'POST'])
def autenticar():
    #método POST - pega nos fields (campos) do formulário
    usuario = request.form.get('nome_usuario')
    senha = request.form.get('senha')
    if usuario == "admin" and senha == "ifro":
        return f"usuario: {usuario} e senha: {senha}"
    else:
        #para não dar msg. na outra página, vamos manter na própria página com flash
        #adicionar import flash
        flash("Dados inválidos!")
        flash("Login ou senha inválidos!")
        return redirect ('/login') #adicionar import redirect
```

```
@app_Mariela.route("/login")
def login():
    return render_template("t_login.html")
```

t_login.html

Para testar um caso válido coloquei um usuário e senha fixo.

LEIA !

```
1  {% extends 'base.html' %}
2
3  {% block conteudo %}
4      <!-- Toda vez que for trabalhar com Python dentro do HTML usa o bloco set
5          variáveis globais (set)
6          variáveis locais (with) e (endwith)-->
7      {% set mensagens = get_flashed_messages() %}
8      {% if mensagens %}
9          <div>
10             <ul>
11                 {% for msg in mensagens %}
12                     <li>{{ msg }}</li>
13                 {% endfor %}
14             </ul>
15          </div>
16      {% endif %}
17
18      <form action="/autenticar" method="POST">
```

RuntimeError: The session is unavailable because no secret key was set.
Set the secret_key on the application to something unique and secret.

Pede para configurar uma chave secreta inicial.

Altera no arquivo .py

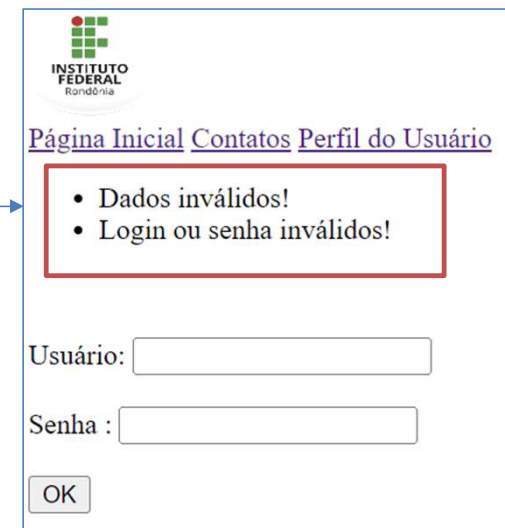
Aqui configura na variável SECRET_KEY e pode atribuir qualquer palavra.

```
# no caso de usar flash pede a configuração de uma chave secreta  
app_Mariela.config['SECRET_KEY'] = "palavra-secreta-IFRO"
```

```
if usuario == "admin" and senha == "ifro":  
    return f"usuario: {usuario} e senha: {senha}"  
else:  
    #para não dar msg. na outra página, vamos manter na própr  
    #adicionar import flash  
    flash("Dados inválidos!")  
    flash("Login ou senha inválidos!")  
    return redirect ('/login') #adicionar import redirect
```

LEIA !

Essas mensagens flash foram repassadas na rota **/login**,
que renderiza no **t_login.html** e 'remonta' (renderiza) a
página que já estava, agora incluindo essas mensagens



INSTITUTO
FEDERAL
Rondônia

[Página Inicial](#) [Contatos](#) [Perfil do Usuário](#)

- Dados inválidos!
- Login ou senha inválidos!

Usuário:

Senha :

OK

Flash vs. Alert (mensagens no html)

Diferença entre flash() e alert() no HTML

A principal diferença entre usar flash no Flask e alert no HTML está no contexto em que são usados e no controle que cada abordagem oferece:

1. Flash no Flask:

- **Finalidade:** Usado para **enviar mensagens temporárias do servidor para o cliente**. É útil em situações em que a **lógica do back-end determina o que deve ser exibido ao usuário**.
- **Funcionamento:**
 - A mensagem é configurada no servidor com `flash("mensagem")`.
 - No lado do cliente (HTML), a mensagem é exibida usando herança de templates e lógica Jinja2, como `{% with messages = get_flashed_messages() %}`.
- **Vantagens:**
 - Perfeito para mensagens baseadas no estado ou eventos no servidor, como "Login bem-sucedido", "Erro ao salvar dados", etc.
 - Persistência de curto prazo: a mensagem permanece disponível até a próxima solicitação.
 - Integrado ao ciclo de vida da aplicação Flask.
- **Desvantagens:** Requer configuração tanto no back-end quanto no front-end. Não é ideal para mensagens instantâneas controladas exclusivamente no lado do cliente. Neste caso use o `alert()` do javascript

2. Alert no HTML (JavaScript):

- **Finalidade:** Exibe uma mensagem imediata no navegador do usuário, geralmente no lado do cliente.
- **Funcionamento:**
 - Criado diretamente no JavaScript com `alert("mensagem");`
 - Não depende de nenhuma interação com o servidor.
- **Vantagens:**
 - Rápido e direto: ideal para mensagens ou avisos simples.
 - Totalmente controlado no lado do cliente.
 - Fácil de implementar.
- **Desvantagens:**
 - Pouco estilizável: o alert nativo tem um design padrão do navegador.
 - Não persiste entre requisições: a mensagem desaparece assim que o usuário interage.

Quando usar Flash ou Alert?

Flash: Use quando a mensagem depende de uma lógica no servidor, como validação de login, resultados de operações no *back-end*, ou para mensagens persistentes de curta duração entre requisições.

Alert: Use para mensagens imediatas que podem ser resolvidas apenas no *front-end*, como validação básica de formulário antes do envio ou avisos simples baseados em eventos no cliente.

Se desejar, pode combinar as duas abordagens: use flash no servidor e exiba a mensagem como um alert no HTML.